

Documento Técnico: Separación Frontend / Backend — Proyecto HeartCoin

Introducción

Este documento describe y analiza de manera detallada el modelo de separación entre el frontend (aplicación cliente) y el backend (servicios de servidor) del proyecto HeartCoin, explicando qué responsabilidades recaen en cada capa, qué mecanismos de comunicación existen entre ambas, y qué principios arquitectónicos determinan dónde se ubica cada pieza de lógica del sistema.

A diferencia de una arquitectura cliente-servidor tradicional con un backend de aplicación propio (por ejemplo, un servidor Node.js o Django intermedio), HeartCoin adopta un modelo de **Backend as a Service (BaaS)**, en el cual la frontera entre frontend y backend no está determinada por la existencia de dos servidores distintos, sino por **quién tiene permiso de hacer qué**, aplicado mediante controles de la propia base de datos y funciones de servidor puntuales.

Objetivo

Documentar con precisión técnica la frontera entre la aplicación cliente (Flutter) y los servicios de servidor (Supabase: PostgreSQL, Auth, Realtime, Edge Functions), explicar los criterios que determinan si una operación se ejecuta del lado del cliente o requiere intervención del servidor, y catalogar de forma exhaustiva los puntos de la aplicación donde esta frontera se materializa.

Alcance

Este documento cubre la totalidad de los mecanismos de comunicación entre el cliente Flutter y Supabase, el catálogo completo de funciones de servidor (RPC y Edge Functions) existentes en el proyecto, y el modelo de seguridad que sostiene la separación de responsabilidades. No cubre aspectos de infraestructura de hosting de Supabase en sí (fuera del control directo del proyecto) ni el diseño visual de la aplicación cliente.

1. Principio Arquitectónico General

HeartCoin **no tiene un servidor de aplicación propio** en el sentido tradicional. El "backend" del proyecto es Supabase en su totalidad: una base de datos PostgreSQL con capacidades extendidas (autenticación, tiempo real, almacenamiento, funciones serverless). La aplicación Flutter se conecta a este backend de dos formas:

1. **Conexión directa** al backend mediante el SDK oficial (`supabase_flutter`), usando una clave pública (`anon key`) diseñada para vivir en el cliente sin riesgo, porque la seguridad no depende de

mantenerla en secreto.

2. **Conexión indirecta** a través de funciones de servidor (Edge Functions) para el conjunto acotado de operaciones que requieren credenciales privilegiadas o lógica de negocio que no puede validarse únicamente con reglas declarativas de base de datos.

Esta decisión arquitectónica traslada la responsabilidad principal de "dónde vive la lógica de negocio crítica" desde un servidor de aplicación hacia la propia base de datos (mediante Row Level Security, triggers y funciones `SECURITY DEFINER`), complementada por un número reducido y bien delimitado de funciones de servidor para los casos que exceden esa capacidad declarativa.

2. Responsabilidades del Frontend (Flutter)

El cliente Flutter es responsable de:

- **Presentación e interacción de usuario:** renderizado de pantallas, navegación, formularios, validación de entrada a nivel de experiencia de usuario (no de seguridad).
- **Orquestación de llamadas al backend:** cada dominio funcional cuenta con una clase de servicio dedicada (patrón singleton `ServiceName.instance`) que encapsula las llamadas al SDK de Supabase — por ejemplo, `WalletService`, `FollowService`, `BlockService`, `CheckinService`.
- **Consumo de datos en tiempo real:** suscripción a cambios en tablas mediante `.stream()` del SDK, utilizado en el feed de publicaciones, notificaciones, saldo de HC y comentarios, entre otros.
- **Estado de sesión local:** lectura del rol y estado de autenticación del usuario desde los metadatos de la sesión (`user_metadata`), sin necesidad de una llamada de red adicional.
- **Persistencia de preferencias locales no sensibles:** tamaño de texto, preferencia de uso de ubicación, mediante `shared_preferences` del dispositivo, sin involucrar al backend.

Importante: el cliente **nunca contiene credenciales privilegiadas**. La única clave presente en el código de la aplicación es la `anon` key de Supabase, cuya exposición pública es una característica del diseño, no una vulnerabilidad — su alcance está limitado por las políticas de seguridad de la base de datos, no por su secretismo.

3. Responsabilidades del Backend (Supabase)

El backend, en su totalidad gestionado por Supabase, es responsable de:

- **Persistencia y modelo de datos:** la base de datos PostgreSQL contiene la totalidad del esquema relacional del sistema.
- **Autorización a nivel de fila:** mediante Row Level Security (RLS), determinando qué usuario puede leer, insertar, actualizar o eliminar cada fila de cada tabla, sin depender de que el cliente respete ninguna regla voluntariamente.

- **Consistencia derivada:** mediante triggers de PostgreSQL, que mantienen contadores, saldos y notificaciones sincronizados como efecto secundario de operaciones primarias, sin que el cliente deba calcularlos ni escribirlos directamente.
- **Validación de reglas de negocio no expresables mediante RLS:** mediante funciones `SECURITY DEFINER` invocadas vía RPC, para casos como validación de geolocalización, ventanas temporales o suficiencia de saldo.
- **Operaciones privilegiadas y comunicación con terceros:** mediante Edge Functions, para los casos que requieren una credencial que jamás debe estar accesible desde el cliente (claves de servicio, secretos de integraciones externas).
- **Autenticación:** emisión, renovación y revocación de sesiones de usuario (Supabase Auth).

4. Mecanismos de Comunicación entre Capas

Mecanismo	Protocolo	Uso
PostgREST	REST autogenerado (<code>/rest/v1/<tabla></code>)	Lectura/escritura directa de tablas desde el cliente, sujeta a RLS
RPC	REST (<code>/rest/v1/rpc/<función></code>)	Invocación de funciones <code>SECURITY DEFINER</code> para reglas de negocio complejas
Realtime	WebSocket	Suscripción en vivo a cambios de tablas habilitadas en la publicación <code>supabase_realtime</code>
Edge Functions	REST (<code>/functions/v1/<nombre></code>)	Operaciones que requieren credenciales de servidor o comunicación con servicios externos
Supabase Auth	REST (gestionado por el SDK)	Registro, inicio de sesión, renovación y cierre de sesión

La aplicación cliente utiliza estos cinco mecanismos de forma transparente a través del SDK `supabase_flutter`, sin que el desarrollador deba construir peticiones HTTP manualmente en la mayoría de los casos (con la excepción de las llamadas a Edge Functions con cuerpo `multipart/form-data`, como la subida de archivos).

5. Criterio de Decisión: ¿Cuándo una Operación Requiere el Backend Activo?

El proyecto sigue, de forma consistente, un criterio de tres condiciones para determinar si una operación debe canalizarse a través de una función de servidor en lugar de una escritura directa del cliente sujeta a RLS:

1. **¿La operación requiere una credencial que no debe existir en el cliente?** (clave de servicio de Supabase, secretos de servicios externos). Si sí → Edge Function.
2. **¿La regla de negocio requiere validar condiciones que RLS no puede expresar declarativamente?** (distancia geográfica, ventanas de tiempo, suficiencia de saldo verificada de forma atómica). Si sí → función RPC `SECURITY DEFINER`.
3. **¿La operación es una lectura o escritura simple restringible por propiedad de fila?** (un usuario edita su propio perfil, lee sus propias notificaciones). Si sí → escritura directa del cliente vía PostgreSQL, protegida únicamente por RLS.

Este criterio explica por qué, por ejemplo, el registro de un check-in en una iniciativa **no** tiene una política de `INSERT` directa para el cliente (depende de geolocalización y ventana horaria, condición 2), mientras que la edición del propio perfil **sí** la tiene (condición 3).

6. Catálogo de la Frontera: Funciones RPC (`SECURITY DEFINER`)

Función	Justificación de su existencia en el backend
<code>checkin_iniciativa</code>	Valida distancia geográfica (haversine) y ventana horaria del evento antes de acreditar HC; un cliente modificado podría falsificar estas condiciones si la validación ocurriera del lado del cliente
<code>redeem_beneficio</code>	Valida estado activo, límite de canjes y suficiencia de saldo de forma centralizada antes de descontar HC
<code>redeem_servicio</code>	Misma lógica que <code>redeem_beneficio</code> , aplicada al catálogo de servicios empresariales

Todas estas funciones utilizan `auth.uid()` internamente, de modo que el cliente nunca transmite explícitamente de quién es la operación — el servidor lo determina a partir de la sesión autenticada, eliminando la posibilidad de que un cliente modificado ejecute una acción en nombre de otro usuario.

7. Catálogo de la Frontera: Edge Functions

Función	Justificación de su existencia en el backend
<code>upload-media</code>	Firma peticiones hacia MediaAAS con credenciales HMAC que no pueden exponerse en el cliente
<code>delete-account</code>	Requiere la clave de servicio de Supabase (<code>auth.admin.deleteUser</code>) para eliminar una cuenta completa
<code>originalauth-register /</code> <code>originalauth-verify-email /</code> <code>originalauth-resend-verification</code>	Requieren la API key de Original Auth (credencial de servidor) para validar/registrar correo electrónico
<code>originalpay-balance / originalpay-redeem</code>	Exponen consulta de saldo y cobro de HC a un tercero externo (Original Pay), autenticado mediante firma HMAC con una API key emitida por HeartCoin, nunca expuesta al cliente móvil
<code>originalpay-webhook</code>	Recibe notificaciones asíncronas de Original Pay; requiere la clave de servicio para ajustar el ledger de HC en caso de reembolso

Se observa un patrón consistente: **toda Edge Function existe porque involucra una credencial de servidor**, ya sea propia de Supabase (clave de servicio) o de un servicio externo (MediaAAS, Original Auth, Original Pay). Ninguna Edge Function del proyecto existe únicamente por conveniencia de organización de código — cada una está justificada por un requisito de seguridad concreto.

8. Modelo de Seguridad que Sostiene la Separación

La separación frontend/backend en HeartCoin no se sostiene mediante ofuscación ni mediante la suposición de que el cliente se comportará correctamente. Se sostiene mediante controles verificables del lado del servidor:

- **Row Level Security** (47 políticas activas dependientes de `auth.uid()`), que se aplican sin importar qué cliente —oficial o modificado— realice la petición.
- **Ausencia deliberada de políticas de INSERT/UPDATE de escritura directa** en tablas cuya integridad depende de una validación de negocio (`iniciativa_checkins`, `beneficio_redemptions`, `servicio_redemptions`, `hc_transactions`), forzando el paso obligatorio por una función de servidor.
- **Inyección automática de la clave de servicio** (`SUPABASE_SERVICE_ROLE_KEY`) en el entorno de ejecución de cada Edge Function por parte de la plataforma, evitando su gestión manual y su posible filtración por configuración incorrecta.

- **Verificación de sesión de usuario dentro de cada Edge Function sensible**, mediante un cliente anon con el token de autorización recibido, antes de proceder con cualquier operación privilegiada — de modo que ni siquiera conociendo la URL de la función es posible invocarla en nombre de otro usuario sin su token de sesión válido.

Fortalezas del Modelo

- **Superficie de ataque reducida**: al no existir un servidor de aplicación propio, se elimina una categoría completa de vulnerabilidades asociadas a la gestión de dicho servidor (parcheo, configuración, exposición de puertos).
- **Consistencia**: la seguridad no depende de que cada desarrollador recuerde replicar validaciones en múltiples capas; las políticas de RLS se aplican de forma uniforme sin importar el punto de entrada.
- **Trazabilidad de la frontera**: el criterio de tres condiciones (sección 5) permite auditar rápidamente si una nueva funcionalidad está correctamente ubicada, simplemente respondiendo a esas tres preguntas.

Riesgos y Áreas de Atención

- **Multiplicación de Edge Functions ante el crecimiento de integraciones externas**: actualmente existen ocho funciones de servidor; cada nueva integración con un tercero añadirá, siguiendo el patrón actual, al menos una función adicional. Sin una capa de abstracción compartida más allá de los módulos `_shared/` ya existentes, este crecimiento podría derivar en duplicación de lógica de manejo de errores y autenticación.
- **Dependencia total de la disponibilidad de Supabase**: al no existir un backend alternativo, cualquier interrupción del servicio de Supabase afecta tanto el frontend como el backend simultáneamente, sin posibilidad de degradación parcial.
- **Ausencia de pruebas automatizadas sobre la frontera**: no se identificaron pruebas que verifiquen explícitamente que las políticas de RLS y las funciones de servidor rechazan correctamente los casos no autorizados, dependiendo actualmente de revisión manual.

Recomendaciones

1. Mantener el criterio de tres condiciones (sección 5) documentado y como referencia obligatoria al evaluar dónde ubicar nueva lógica de negocio.
2. Evaluar la extracción de un módulo compartido adicional para el manejo estandarizado de errores y respuestas HTTP entre Edge Functions, dado el crecimiento observado en su número.
3. Incorporar pruebas automatizadas que verifiquen negativamente las políticas de RLS (es decir, que confirmen que un usuario no puede leer o modificar datos que no le pertenecen), no únicamente los

casos de éxito.

Conclusión

La separación entre frontend y backend en HeartCoin no se basa en la existencia de dos servidores físicamente distintos, sino en un modelo de **autorización declarativa a nivel de base de datos**, complementado por un conjunto reducido y consistentemente justificado de funciones de servidor para los casos que exceden esa capacidad declarativa. Este modelo, propio de una arquitectura Backend as a Service, ha sido aplicado con disciplina a lo largo del proyecto: cada punto en que la lógica se traslada al servidor responde a un criterio de seguridad identificable y verificable, no a una decisión arbitraria de organización de código.